



XML-RPC



Reti di Calcolatori Modulo 2

Introduction

- ▶ XML-RPC provides an XML- and HTTP-based mechanism for making method or function calls across a network
 - ▶ Use **XML** for messaging (use only a small XML vocabulary set)
 - ▶ Use **HTTP** to pass information from a client computer to a server computer
- ▶ No notion of objects
- ▶ No mechanism for including information that uses other XML vocabularies
- ▶ Emerged in early 1998

Introduction

- ▶ As distributed systems become more common, there is a need **to integrate the applications** in different computers, even within a company
- ▶ XML-RPC provides a simple tool to connect disparate parts inside a private network and a simple interface to a computer for other computers in the world to access
 - ▶ XML **simple to marshal**
 - ▶ XML has **broad support**
- ▶ Similar to the case that a Web page is a human readable interface of a computer

Main components

- ▶ Data Model
 - ▶ A set of types for passing parameters, return values, and faults (error messages)
 - ▶ Used by both request and response
- ▶ Request Structure
 - ▶ An HTTP post request containing method name and parameter information
- ▶ Response Structure
 - ▶ An HTTP response containing return values or fault information

Data Model

- ▶ Define only 6 basic data types and 2 compound data types
- ▶ Seems to be enough for most practical applications

Basic Type names	Description
int or i4	32-bit integers
double	64-bit floating pt
boolean	true(1) or false(0)
string	ASCII text, usually support Unicode
dateTime.iso8601	Dates in ISO8601 format: CCYYMMDDTHH:MM:SS
base64	Binary defined as in RFC2045

Data Model: Examples

- ▶ `<int>27</int>`
- ▶ `<double>27.31415</double>`
- ▶ `<boolean>1</boolean>`
- ▶ `<string>Hello</string>`
- ▶ `<dateTime.iso8601>`
 `20020125T02:20:04`
 `</dateTime.iso8601>`
- ▶ `<base64>SGVsbG8sIFdvcmxkIQ==</base64>`
- ▶ Basic types always enclosed in an element value
 `<value>`
 `<double> -12.45 </double>`
 `</value>`

Data Model: Complex Types

- ▶ These basic types can be combined into two more complex types: Array and Struct

```
<value>
  <array>
    <data>
      <value><string>This </string></value>
      <value><string>is </string></value>
      <value><string>an </string></value>
      <value><string>array.</string></value>
    </data>
  </array>
</value>
```

Data Model: Arrays

- ▶ Arrays can contain mixtures of different types

```
<value>
  <array>
    <data>
      <value><boolean>1</boolean></value>
      <value><string>Chan Tai-Man </string></value>
      <value><int> -91 </int></value>
      <value><double>0.1234</double></value>
    </data>
  </array>
</value>
```

Data Model: Arrays

- ▶ Arrays can be multi-dimensional
- ▶ E.g.

10	20
30	40

```
<value>
  <array>
    <data>
      <value>
        <array>
          <data>
            <value><int>10</int></value>
            <value><int>20</int></value>
          </data>
        </array>
      </value>
    <value>
      <array>
        <data>
          <value><int>30</int></value>
          <value><int>40</int></value>
        </data>
      </array>
    </value>
  </data>
</array>
</value>
```

Data Model: Structs

- ▶ Struct contains unordered content, identified by name
- ▶ Names are string, although it is not necessary to enclose them by string elements
- ▶ Each struct element contains a list of member elements
- ▶ Member elements each contain one name element and one value element
- ▶ The order of members is not important

Data Model: Structs

```
<value>
  <struct>
    <member>
      <name>givenName</name>
      <value><string>Tai-Man </string></value>
    </member>
    <member>
      <name>familyName</name>
      <value><string>Chan </string></value>
    </member>
    <member>
      <name>age</name>
      <value><int>27</int></value>
    </member>
  </struct>
</value>
```

First element

Second element

Third element

Data Model: Array and Structs

- ▶ We can nest struct/arrays inside structs

```
<value>
  <struct>
    <member>
      <name>Name</name>
      <value><string>a</string></value>
    </member>
    <member>
      <name>attributes</name>
      <value><struct>
        <member><name>href</name>
        <value><string>http://ex.com</string></value>
      </member>
      <member><name>target</name>
      <value><string>_top</string></value>
    </member>
      </struct></value>
    </member>
  </struct>
</value>
```

Request Structure

- ▶ XML-RPC requests are a combination of XML content and HTTP headers
 - ▶ HTTP header: a wrapper for passing the request over the Web
 - ▶ XML content: pass parameters and identify the procedure to be called

POST /xmlrpc HTTP 1.0
User-Agent: myXMLRPCClient/1.0
Host: 192.168.124.2
Content-Type: text/html
Content-Length: 169

:
XML statements
:

Request Structure

- ▶ The XML statements contain the method name and passed parameters
 - ▶ Method to be called: circleArea
 - ▶ Input parameters
 - ▶ A double
 - ▶ An array

```
<?xml version="1.0"?>
<methodCall>
  <methodName>circleArea</methodName>
  <params>
    <param>
      <value><double>2.42</double></value>
    </param>
    <param>
      <value>
        <array>
          <data>
            <value><int>10</int></value>
            <value><int>20</int></value>
          </data>
        </array>
      </value>
    </param>
  </params>
</methodCall>
```

Response Structure

- ▶ If a request is successful – the procedure was found, executed correctly, the result will be returned through the response to the client
- ▶ Similar to request, a response needs to be attached to a HTTP header for surfing on the Web

HTTP/1.1 200 OK

Date: Sat, 06 Oct 2004 23:20:04 GMT

Server: Apache.1.3.12 (Unix)

Connection: close

Content-Type: text/html

Content-Length: 124

:

XML statements

:

Response Structure

```
<?xml version="1.0"?>
<methodResponse>
  <params>
    <param>
      <value>
        <double>2.4</double>
      </value>
    </param>
  </params>
</methodResponse>
```

► Return a Double

```
<?xml version="1.0"?>
<methodResponse>
  <fault>
    <value><string>No such method!</string></value>
  </fault>
</methodResponse>
```

► Return an error

► No standardized errors

Developing with XML-RPC

- ▶ In real applications, we do not need to directly program the XML statements
- ▶ Only need to add a XML-RPC library and making some function calls through this library
- ▶ Hence it can be done by using any programming languages, such as Java
 - ▶ The Apache XML Project's Apache XML-RPC has provided packages that make integrating XML-RPC with Java easier (<http://ws.apache.org/xmlrpc/>)
 - ▶ XML RPC for C/C++ (<http://xmlrpc-c.sourceforge.net/>)

Advantages

- ▶ Simplicity
- ▶ Common format for request and response
- ▶ Standard protocols
- ▶ Standard specifications

Disadvantages

- ▶ XML-RPC is impoverished in the type of data it can transmit and obese in its message size
- ▶ XML-RPC transmits messages lacking statefulness and incurs channel bottlenecks
- ▶ Compared to SOAP, XML-RPC lacks both important security mechanisms and a robust object model
- ▶ As a data representation, XML-RPC is slow, cumbersome, and incomplete compared to native programming language mechanisms like Java

Conclusions

- ▶ XML-RPC provides an XML- and HTTP-based mechanism for making method or function calls across a network
- ▶ For XML-RPC, client needs to have some knowledge of server's registered functions
- ▶ No way for the server to publicize its functions or to allow the client to automatically obtain such info
 - ▶ Hence it does not perfectly fit the requirements of Web Services (SOAP-based web services)
- ▶ Lacks of security